UNIT III

SOFTWARE DESIGN

Software design – Design process – Design concepts – Coupling – Cohesion – Functional independence – Design patterns – Model-view-controller – Publish-subscribe – Adapter – Command– Strategy – Observer – Proxy – Facade – Architectural styles – Layered - Client Server - Tiered -Pipe and filter- User interface design-Case Study.

3.1 Software Design

Software design is a mechanism to transform user requirements into finished software product, which helps the programmer in software coding and implementation. It deals with representing the client's requirement, as described in SRS (Software Requirement Specification) document, into a form, i.e., easily implementable using programming language. Software design phase is the first step in SDLC (Software Design Life Cycle), which moves the concentration from the problem domain to the solution domain.

3.1.1 Objectives of Software Design

1. Correctness: Software design should be correct as per requirement.
2. Completeness: The design should have all components like data structures, modules, and external interfaces, etc.
3. Efficiency: Resources should be used efficiently by the program.
4. Flexibility: Able to modify on changing needs.
5. Consistency: There should not be any inconsistency in the design.
6. Maintainability: The design should be so simple so that it can be easily maintainable by other designers.

Objectives of Software Design

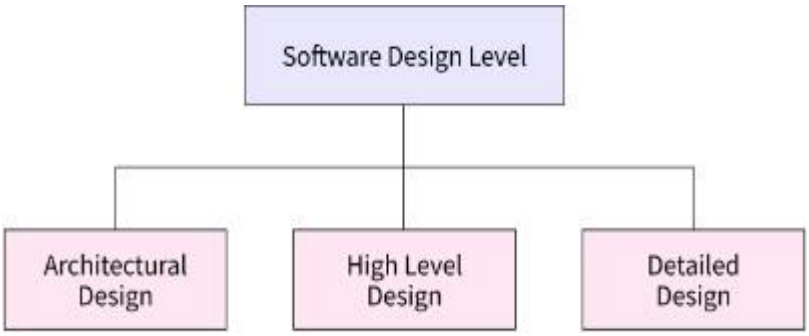
3.1.2 Levels of Software Design

There are three levels of software design.

i. Architectural Design

A system's architecture can be defined as the system's overall structure and how that structure offers conceptual integrity to the system. The architectural design characterizes the software as a system with numerous interconnected components.

The designers acquire an overview of the proposed solution domain at this level.



ii. High level Design

The high-level design deconstructs the architectural design's 'single entity-multiple component' notion into a less abstract perspective of subsystems and modules, depicting their interaction with one another. High-level design is concerned with how the system and its components can be implemented as modules. It recognizes the modular structure of each subsystem, as well as their relationship and interaction with one another.

iii. Detailed Design

After the high-level design is completed, the detailed design begins. Each module is extensively investigated at this level of software design to establish the data structures and algorithms to be used. Finally, a module specification document is used to document the stage's outcome. It defines the logical structure of each module as well as its interfaces with other modules.

3.2 Design Process

Software design is an iterative process through which requirements are translated into a “blueprint” for constructing the software. Initially, the blueprint depicts a holistic view of software. That is, the design is directly traced to the specific system objective and more detailed data, functional, and behavioral requirements.

3.2.1 Software Quality Guidelines and Attributes

Three characteristics that serve as a guide for the evaluation of a good design are

- The design must implement all of the explicit requirements contained in the requirements model, and it must accommodate all of the implicit requirements desired by stakeholders.

- The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

The following are the various quality Guidelines:

1. A design should exhibit an architecture that
   a) has been created using recognizable architectural styles or patterns,
   b) is composed of components that exhibit good design characteristics and
   c) can be implemented in an evolutionary fashion, thereby facilitating implementation and testing.
2. A design should be modular; that is, the software should be logically partitioned into elements or subsystems.
3. A design should contain distinct representations of data, architecture, interfaces, and components.
4. A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.
5. A design should lead to components that exhibit independent functional characteristics.
6. A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.
7. A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.
8. A design should be represented using a notation that effectively communicates its meaning.

3.2.2 Quality Attributes

A set of software quality attributes that has been given the acronym FURPS.

FURPS refers to functionality, usability, reliability, performance, and supportability. The FURPS quality attributes represent a target for all software design:

- **Functionality** is assessed by evaluating the feature set and capabilities of the program, the generality of the functions that are delivered, and the security of the overall system.

- **Usability** is assessed by considering human factors, overall aesthetics, consistency, and documentation.
- **Reliability** is evaluated by measuring the frequency and severity of failure, the accuracy of output results, the mean-time-to-failure (MTTF), the ability to recover from failure, and the predictability of the program.
- **Performance** is measured by considering processing speed, response time, resource consumption, throughput, and efficiency.
- **Supportability** combines the ability to extend the program (extensibility), adaptability, serviceability, these three attributes represent a more common term, maintainability and in addition, testability, compatibility, configurability (the ability to organize and control elements of the software configuration), the ease with which a system can be installed, and the ease with which problems can be localized

3.3 Design Concepts

Fundamental software design concepts provide the necessary framework for a software engineer for the traditional and object-oriented software development. They are,

3.3.1 Abstraction

Abstraction refers to a powerful design tool, which allows software designers to consider components at an abstract level, while neglecting the implementation details of the components. IEEE defines abstraction as 'a view of a problem that extracts the essential information relevant to a particular purpose and ignores the remainder of the information.' The lower level of abstraction provides a more detail description of the solution.

i.A data abstraction is a named collection of data that describes a data object.

ii.A procedural abstraction refers to a sequence of instructions that have a specific and limited function. The name of a procedural abstraction implies these functions, but specific details are suppressed.

3.3.2 Architecture

The complete structure of the software is known as software architecture. Structure provides conceptual integrity for a system in a number of ways. The architecture is the structure of program modules where they interact with each other in a specialized way. The components use the structure of data.

A set of properties as part of an architectural design:

i. **Structural properties -**This aspect of the architectural design representation defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another.

ii. **Families of related systems**-The architectural design should draw upon repeatable patterns that are commonly encountered in the design of families of similar systems.

iii. **Extra-functional properties**-The architectural design description should address how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.

3.3.3 Patterns

A design pattern describes a design structure that solves a particular design problem within a specific context. The intent of each design pattern is to provide a description that enables a designer to determine

i. whether the pattern is applicable to the current work,

ii. whether the pattern can be reused (hence, saving design time), and

iii. whether the pattern can serve as a guide for developing a similar, but functionally or structurally different pattern.

3.3.4 Separation of Concerns

A concern is a feature or behavior that is specified as part of the requirements model for the software.

By separating concerns into smaller and therefore more manageable pieces, a problem takes less effort and time to solve.

3.3.5 Modularity

Modularity is the most common manifestation of separation of concerns. Software is divided into separately named and addressable components, sometimes called modules that are integrated to satisfy problem requirements. It has been stated that “modularity is the single attribute of software that allows a program to be intellectually manageable” .Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer.

As the number of modules grows, the effort (cost) associated with integrating the modules also grow. These characteristics lead to a total cost or effort curve is shown in the below figure.

There is a number, M, of modules that would result in minimum development cost, but we do not have the necessary sophistication to predict M with assurance.

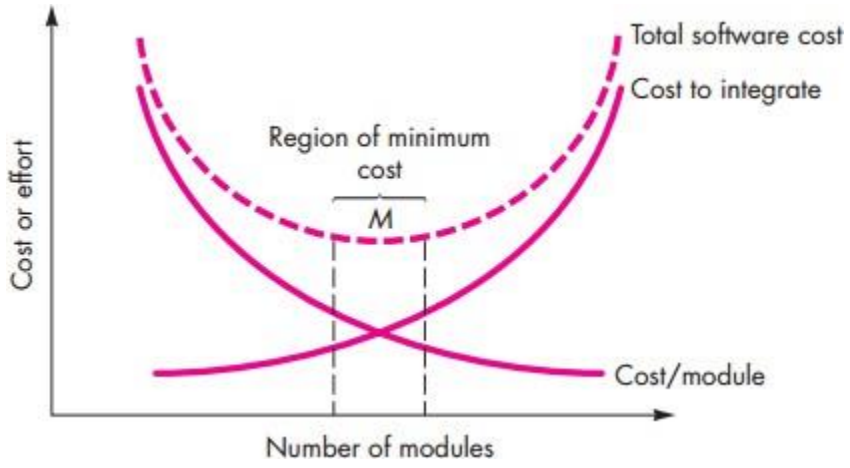


Figure 3.1: Modularity and software cost

3.3.6 Information Hiding

Modules should be specified and designed in such a way that the data structures and processing details of one module are not accessible to other modules. They pass only that much information to each other, which is required to accomplish the software functions. The way of hiding unnecessary details is referred to as information hiding. IEEE defines information hiding as 'the technique of encapsulating software design decisions in modules in such a way that the module's interfaces reveal as little as possible about the module's inner workings; thus each module is a 'black box' to the other modules in the system.

Some of the advantages associated with information hiding are listed below.

- Leads to low coupling
- Emphasizes communication through controlled interfaces
- Decreases the probability of adverse effects
- Restricts the effects of changes in one component on others
- Results in higher quality software.

3.3.7 Functional Independence

The functional independence is the concept of separation and related to the concept of modularity, abstraction and information hiding. The functional independence is accessed using two criteria i.e Cohesion and coupling.

- Cohesion

Cohesion is an extension of the information hiding concept. A cohesive module performs a single task and it requires a small interaction with the other components in other parts of the program.

- Coupling

Coupling is an indication of interconnection between modules in a structure of software.

3.3.8 Refinement

Refinement is a top-down design approach. It is a process of elaboration. A program is established for refining levels of procedural details. A hierarchy is established by decomposing a statement of function in a stepwise manner till the programming language statement are reached.

3.3.9 Refactoring

"Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure." When software is refactored, the existing design is examined for redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failure that can be corrected to yield a better design.

3.3.10 Object-Oriented Design Concepts

Object-oriented concepts are used in the design methods such as classes, objects, polymorphism, encapsulation, inheritance, dynamic binding, information hiding, interface, constructor, and destructor.

The main advantage of object oriented design is that improving the software development and maintainability. Another advantage is that faster and low cost development, and creates high quality software.

3.3.11 Design Classes

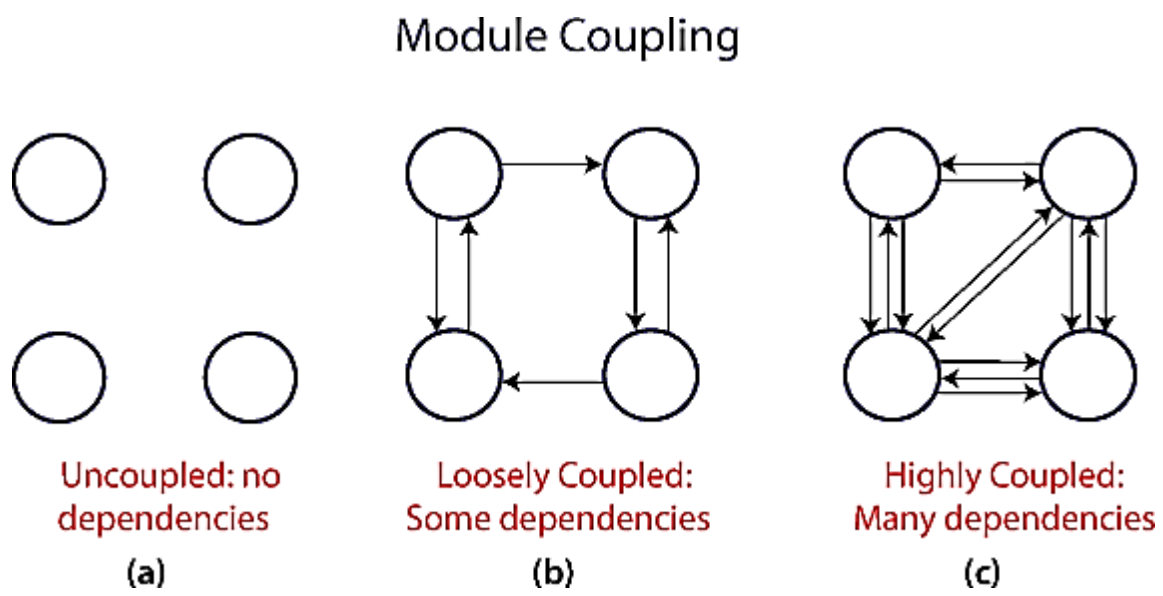
There are five different types of design classes and each type represents the layer of the design architecture these are as follows:

- User interface classes-These classes are designed for Human Computer Interaction (HCI). These interface classes define all abstraction which is required for Human Computer Interaction (HCI).
- Business domain classes-These classes are commonly refinements of the analysis classes. These classes are recognized as attributes and methods which are required to implement the elements of the business domain.
- Process classes-It implements the lower level business abstraction which is needed to completely manage the business domain class.
- Persistence classes-It shows data stores that will persist behind the execution of the software.

3.4 Module Coupling

In software engineering, the coupling is the degree of interdependence between software modules. Two modules that are tightly coupled are strongly dependent on each other. However, two modules that are loosely coupled are not dependent on each other. .

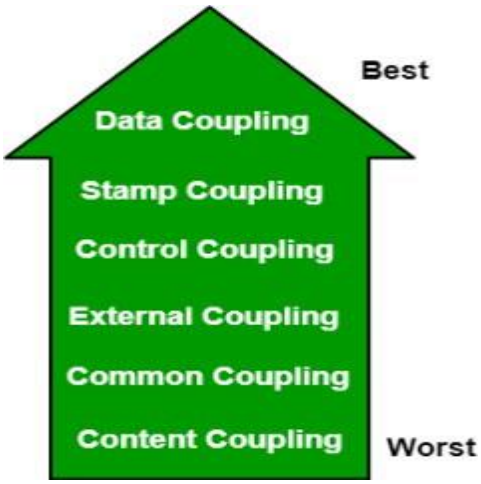
The various types of coupling techniques are shown in figure:



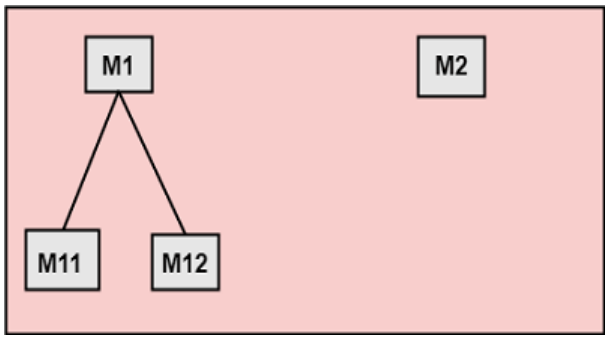
A good design is the one that has low coupling. Coupling is measured by the number of relations between the modules. That is, the coupling increases as the number of calls between modules

increase or the amount of shared data is large. Thus, it can be said that a design with high coupling will have more errors.

Types of Module Coupling

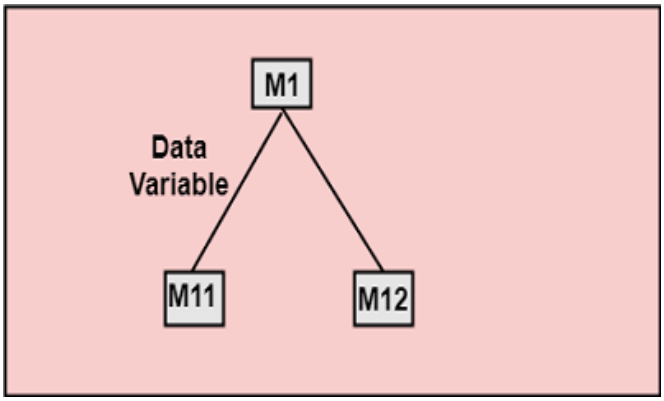


1. No Direct Coupling: There is no direct coupling between M1 and M2.



In this case, modules are subordinates to different modules. Therefore, no direct coupling.

2. Data Coupling: When data of one module is passed to another module, this is called data coupling.

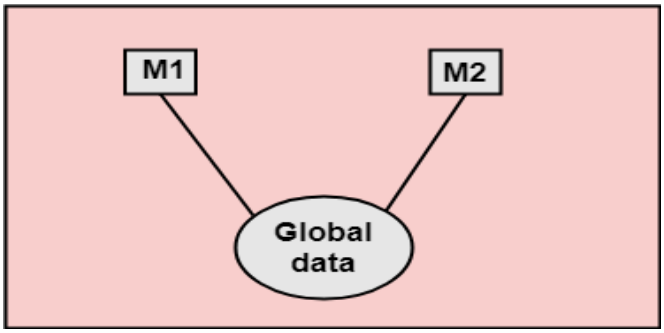


3. Stamp Coupling: Two modules are stamp coupled if they communicate using composite data items such as structure, objects, etc. When the module passes non-global data structure or entire structure to another module, they are said to be stamp coupled. For example, passing structure variable in C or object in C++ language to a module.

4. Control Coupling: Control Coupling exists among two modules if data from one module is used to direct the structure of instruction execution in another.

5. External Coupling: External Coupling arises when two modules share an externally imposed data format, communication protocols, or device interface. This is related to communication to external tools and devices.

6. Common Coupling: Two modules are common coupled if they share information through some global data items.



7. Content Coupling: Content Coupling exists among two modules if they share code, e.g., a branch from one module into another module.

Advantages of low coupling:

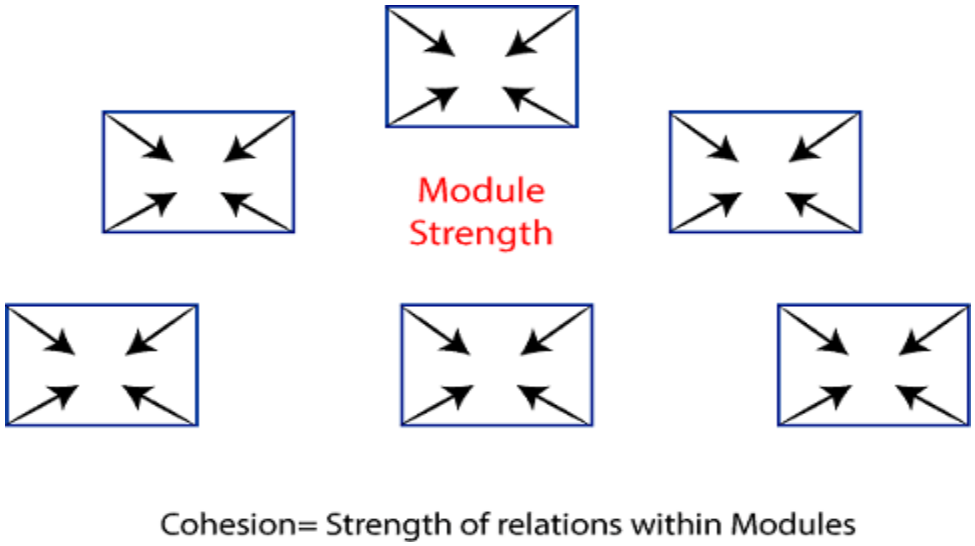
- Improved maintainability: Low coupling reduces the impact of changes in one module on other modules, making it easier to modify or replace individual components without affecting the entire system.
- Enhanced modularity: Low coupling allows modules to be developed and tested in isolation, improving the modularity and reusability of code.
- Better scalability: Low coupling facilitates the addition of new modules and the removal of existing ones, making it easier to scale the system as needed.

3.5 Module Cohesion

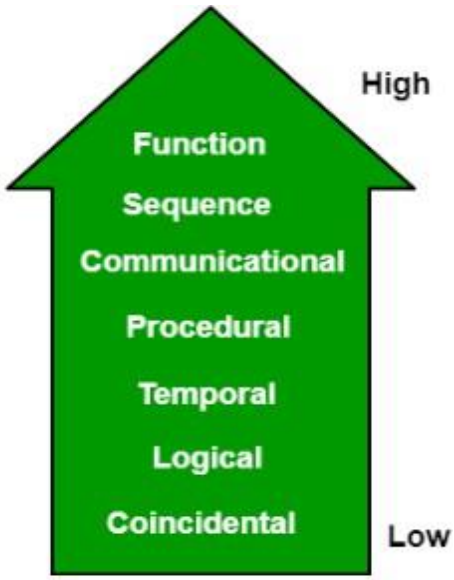
In computer programming, cohesion defines to the degree to which the elements of a module belong together. Thus, cohesion measures the strength of relationships between pieces of

functionality within a given module. For example, in highly cohesive systems, functionality is strongly related.

Cohesion is an **ordinal** type of measurement and is generally described as "high cohesion" or "low cohesion."



Types of Modules Cohesion



1. **Functional Cohesion:** Functional Cohesion is said to exist if the different elements of a module, cooperate to achieve a single function.

2. **Sequential Cohesion:** A module is said to possess sequential cohesion if the element of a module form the components of the sequence, where the output from one component of the sequence is input to the next.
3. **Communicational Cohesion:** A module is said to have communicational cohesion, if all tasks of the module refer to or update the same data structure, e.g., the set of functions defined on an array or a stack.
4. **Procedural Cohesion:** A module is said to be procedural cohesion if the set of purpose of the module are all parts of a procedure in which particular sequence of steps has to be carried out for achieving a goal, e.g., the algorithm for decoding a message.
5. **Temporal Cohesion:** When a module includes functions that are associated by the fact that all the methods must be executed in the same time, the module is said to exhibit temporal cohesion.
6. **Logical Cohesion:** A module is said to be logically cohesive if all the elements of the module perform a similar operation. For example Error handling, data input and data output, etc.
7. **Coincidental Cohesion:** A module is said to have coincidental cohesion if it performs a set of tasks that are associated with each other very loosely, if at all.

Advantages of high cohesion:

- Improved readability and understandability: High cohesion results in clear, focused modules with a single, well-defined purpose, making it easier for developers to understand the code and make changes.
- Better error isolation: High cohesion reduces the likelihood that a change in one part of a module will affect other parts, making it easier to
- isolate and fix errors. Improved reliability: High cohesion leads to modules that are less prone to errors and that function more consistently,
- leading to an overall improvement in the reliability of the system.

Disadvantages of high coupling:

- Increased complexity: High coupling increases the interdependence between modules, making the system more complex and difficult to understand.
- Reduced flexibility: High coupling makes it more difficult to modify or replace individual components without affecting the entire system.

- Decreased modularity: High coupling makes it more difficult to develop and test modules in isolation, reducing the modularity and reusability of code.

Disadvantages of low cohesion:

- Increased code duplication: Low cohesion can lead to the duplication of code, as elements that belong together are split into separate modules.
- Reduced functionality: Low cohesion can result in modules that lack a clear purpose and contain elements that don't belong together, reducing their functionality and making them harder to maintain.
- Difficulty in understanding the module: Low cohesion can make it harder for developers to understand the purpose and behavior of a module, leading to errors and a lack of clarity.

3.6 Design Pattern

A design pattern is the re-usable form of a solution to a commonly occurring problem with in a given context in a software design.

Types of design patterns

There are about 26 Patterns currently discovered (I hardly think I will do them all…).

These 26 can be classified into 3 types:

i. **Creational**: These patterns are designed for class instantiation. They can be either class-creation patterns or object-creational patterns.

ii. **Structural**: These patterns are designed with regard to a class's structure and composition. The main goal of most of these patterns is to increase the functionality of the class(es) involved, without changing much of its composition.

iii. **Behavioral**: These patterns are designed depending on how one class communicates with others.

3.6.1 Design Pattern Template

Pattern name—describes the essence of the pattern in a short but expressive name Problem—describes the problem that the pattern addresses

Motivation—provides an example of the problem Context—describes the environment in which the problem resides including the application domain

Forces—lists the system of forces that affect the manner in which the problem must be solved; includes a discussion of limitations and constraints that must be considered

Solution—provides a detailed description of the solution proposed for the problem Intent—describes the pattern and what it does

Collaborations—describes how other patterns contribute to the solution Consequences—describes the potential trade-offs that must be considered when the pattern is implemented and the consequences of using the pattern

Implementation—identifies special issues that should be considered when implementing the pattern Known uses—provides examples of actual uses of the design pattern in real applications

Related patterns—cross-references related design patterns

i. Creational Patterns

- Abstract factory pattern: centralize decision of what factory to instantiate.
- Factory method pattern: centralize creation of an object of a specific type choosing one of several implementations.
- Builder pattern: separate the construction of a complex object from its representation so that the same construction process can create different representations.
- Prototype pattern: used when the inherent cost of creating a new object in the standard way (e.g., using the "new" keyword) is prohibitively expensive for a given application.
- Singleton pattern: restrict instantiation of a class to one object.

ii. Structural Patterns

- Adapter pattern: "adapts" one interface for a class into one that a client expects.
- Aggregate pattern: a version of the composite pattern with methods for aggregation of children.
- Bridge pattern: decouple an abstraction from its implementation so that the two can vary independently.
- Composite pattern: a tree structure of objects where every object has the same interface.
- Container pattern: create objects for the sole purpose of holding other objects and managing them.
- Proxy pattern: a class functioning as an interface to another thing.
- Pipes and filters: a chain of processes where the output of each process is the input of the next.

iii. Behavioral Patterns

• Chain of responsibility pattern: Command objects are handled or passed on to other objects by logic-containing processing objects.

• Command pattern: Command objects encapsulate an action and its parameters.

• Event listener: Data are distributed to objects that are registered to receive them.

• Interpreter pattern: Implement a specialized computer language to rapidly solve a specific set of problems.

• Iterator pattern: Iterators are used to access the elements of an aggregate object sequentially without exposing its underlying representation.

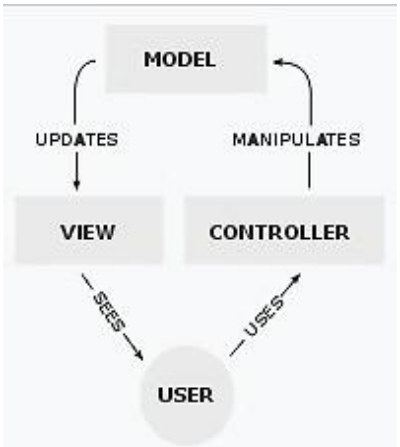
• Mediator pattern: Provides a unified interface to a set of interfaces in a subsystem. • Visitor pattern: A way to separate an algorithm from an object.

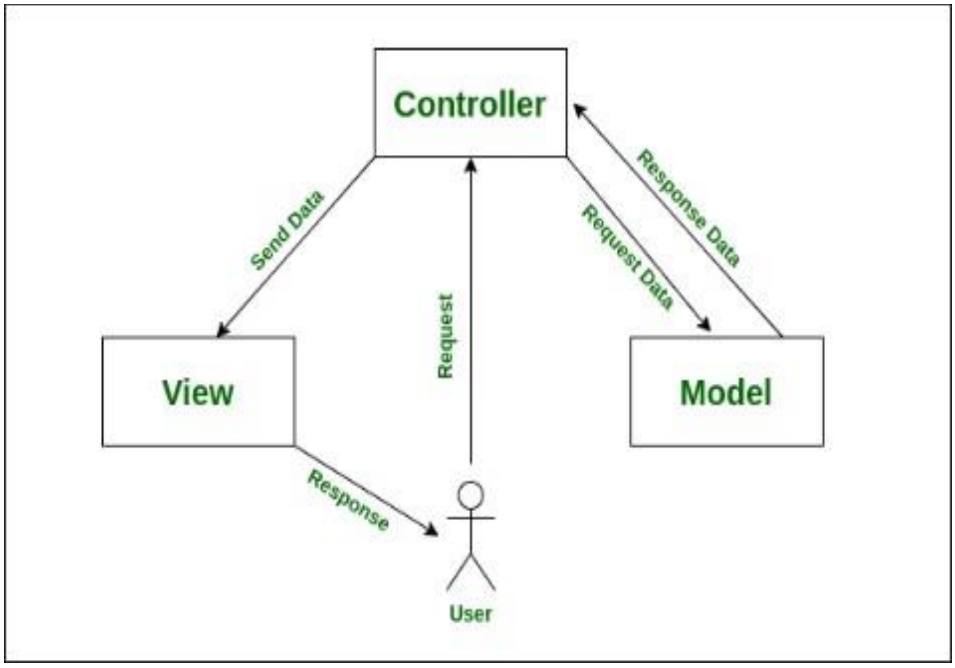
• Single-serving visitor pattern: Optimize the implementation of a visitor that is allocated, used only once, and then deleted.

• Hierarchical visitor pattern: Provide a way to visit every node in a hierarchical data structure such as a tree.

3.7 Model-View-Controller (MVC)

Model–view–controller (MVC) is a software design pattern commonly used for developing user interfaces that divides the related program logic into three interconnected elements. The elements are the internal representations of information (the Model), the interface (the View) that presents information to and accepts it from the user, and the Controller software linking the two.





Traditionally used for desktop graphical user interfaces (GUIs), this pattern became popular for designing web applications.

- The Model contains the pure application data and pure logic describing how to present the data to a user. (Its just a data that is shipped across the application like for example from back-end server view and from front-end view to the database. In java programming,Model can be represented by the use of POJO (Plain-old-java-object) which is a simple java class.
- The View presents the model's data to the user. The view knows how to access the model's data, but it does not know what this data means or what the user can do to manipulate it. View just represent, displays the application's data on screen. View page are generally in the format of .html or .jsp in java programming (which is flexible).
- The Controller exists between the view and the model. It listens to events triggered by the view (or another external source) and executes the appropriate reaction to these events. In most cases, the reaction is to call a method on the model. Since the view and the model are connected through a notification mechanism, the result of this action is then automatically reflected in the view.

In addition to dividing the application into these components, the model–view–controller design defines the interactions between them.

- The model is responsible for managing the data of the application. It receives user input from the controller.
- The view renders presentation of the model in a particular format.
- The controller responds to the user input and performs interactions on the data model objects. The controller receives the input, optionally validates it and then passes the input to the model.

As with other software patterns, MVC expresses the "core of the solution" to a problem while allowing it to be adapted for each system.

Advantages

- Multiple developers can work simultaneously on the model, controller and views.
- MVC enables logical grouping of related actions on a controller together. The views for a specific model are also grouped together.
- Models can have multiple views.

Disadvantages

- The framework navigation can be complex because it introduces new layers of abstraction and requires users to adapt to the decomposition criteria of MVC.
- Knowledge on multiple technologies becomes the norm. Developers using MVC need to be skilled in multiple technologies.

3.8 Publisher-Subscriber pattern

The publish-subscribe (or pub/sub) messaging pattern is a design pattern that provides a framework for exchanging messages that allows for loose coupling and scaling between the sender of messages (publishers) and receivers (subscribers) on topics they subscribe to.Messages are sent (pushed) from a publisher to subscribers as they become available. The host (publisher) publishes messages (events) to channels (topics). Subscribers can sign up for the topics they are interested in.

Also called: Pub/sub messaging

Context and problem: In cloud-based and distributed applications, components of the system often need to provide information to other components as events happen.

Asynchronous messaging is an effective way to decouple senders from consumers, and avoid blocking the sender to wait for a response. However, using a dedicated message queue for each consumer does not effectively scale to many consumers. Also, some of the consumers might be

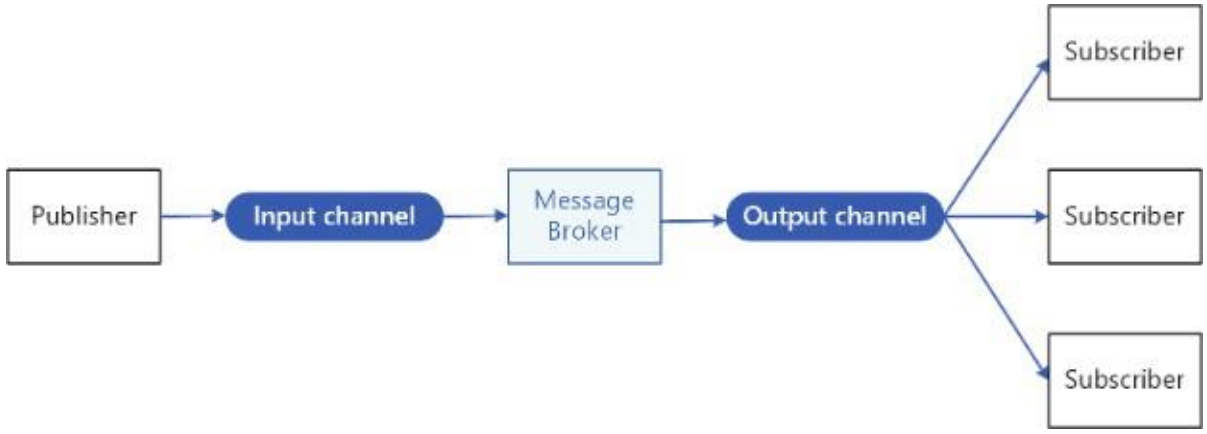
interested in only a subset of the information. How can the sender announce events to all interested consumers without knowing their identities?

Solution:

Introduce an asynchronous messaging subsystem that includes the following:

- An input messaging channel used by the sender. The sender packages events into messages, using a known message format, and sends these messages via the input channel. The sender in this pattern is also called the publisher.
- One output messaging channel per consumer. The consumers are known as subscribers.
- A mechanism for copying each message from the input channel to the output channels for all subscribers interested in that message. This operation is typically handled by an intermediary such as a message broker or event bus.

The following diagram shows the logical components of this pattern:



Pub/sub messaging has the following benefits:

- It decouples subsystems that still need to communicate. Subsystems can be managed independently, and messages can be properly managed even if one or more receivers are offline.
- It increases scalability and improves responsiveness of the sender. The sender can quickly send a single message to the input channel, then return to its core processing responsibilities.

The messaging infrastructure is responsible for ensuring messages are delivered to interested subscribers.

- It improves reliability. Asynchronous messaging helps applications continue to run smoothly under increased loads and handle intermittent failures more effectively.
- It improves testability. Channels can be monitored and messages can be inspected or logged as part of an overall integration test strategy.
- It provides separation of concerns for your applications. Each application can focus on its core capabilities, while the messaging infrastructure handles everything required to reliably route messages to multiple consumers.

Issues and considerations

Consider the following points when deciding how to implement this pattern:

- Existing technologies. It is strongly recommended to use available messaging products and services that support a publish-subscribe model, rather than building your own. Subscription handling. The messaging infrastructure must provide mechanisms that consumers can use to subscribe to or unsubscribe from available channels.
- Security. Connecting to any message channel must be restricted by security policy to prevent eavesdropping by unauthorized users or applications.
- Subsets of messages. Subscribers are usually only interested in subset of the messages distributed by a publisher.
- Wildcard subscribers. Consider allowing subscribers to subscribe to multiple topics via wildcards.
- Bi-directional communication. The channels in a publish-subscribe system are treated as unidirectional. If a specific subscriber needs to send acknowledgment or communicate status back to the publisher, consider using the Request/Reply Pattern. This pattern uses one channel to send a message to the subscriber, and a separate reply channel for communicating back to the publisher.
- Message priority. Some solutions may require that messages are processed in a specific order. The Priority Queue pattern provides a mechanism for ensuring specific messages are delivered before others.

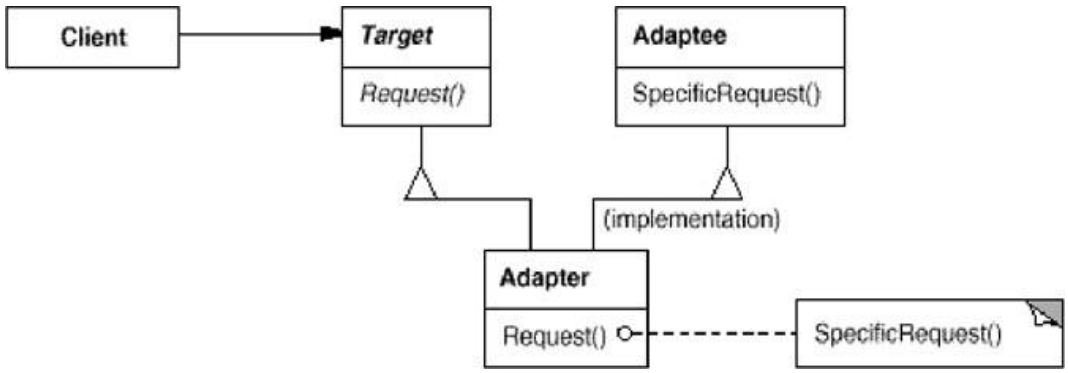
- Repeated messages. The same message might be sent more than once. For example, the sender might fail after posting a message. Then a new instance of the sender might start up and repeat the message. The messaging infrastructure should implement duplicate message detection and removal (also known as de-duping) based on message IDs in order to provide at-most-once delivery of messages. Alternatively, if using messaging infrastructure which doesn't de-duplicate messages, make sure the message processing logic is idempotent.
- Message expiration. A message might have a limited lifetime. If it isn't processed within this period, it might no longer be relevant and should be discarded. A sender can specify an expiration time as part of the data in the message. A receiver can examine this information before deciding whether to perform the business logic associated with the message.

When to use this pattern-Use this pattern when:

- An application needs to broadcast information to a significant number of consumers.
- An application needs to communicate with one or more independently-developed applications or services, which may use different platforms, programming languages, and communication protocols.
- An application can send information to consumers without requiring real-time responses from the consumers.

3.9 Adapter

Intent: Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.



Also Known As: Wrapper

Motivation:Adapter lets classes work together that couldn’t otherwise because of incompatible

interfaces. Adapter is responsible for functionality the adapted class does not provide.

Structure:

Class Adapter (uses multiple interfaces to adapt)

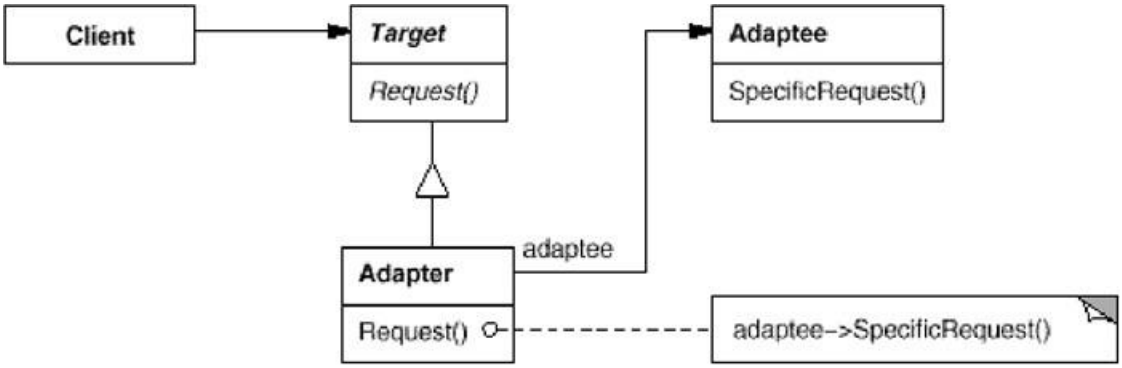
Applicability:

- The existing class, and its interface does not match the one you need.
- to create a reusable class that cooperates with unrelated classes,
- An object adapter can adapt the interface of its parent class.

Participants:

- Target - defines the domain-specific interface that Client
- Client - collaborates with objects conforming to the Target interface.
- Adaptee - defines an existing interface that needs adapting.
- Adapter - adapts the interface of Adaptee to the Target interface

Object Adapter (relies on object composition)



Collaborations: Clients call operations on an Adapter instance. In turn, the adapter calls Adaptee operations that carry out the request

Consequences:

A class adapter

- won't work when we want to adapt a class and all its subclasses.
- lets Adapter override some of Adaptee's behavior, since Adapter is a subclass of Adaptee.

An Object Adapter

- lets a single Adapter work with many Adaptees

- makes it harder to override Adaptee behavior

Implementation:In C++ implementation of a class adapter ,Adapter would inherit publicly from Target and privately from Adaptee.

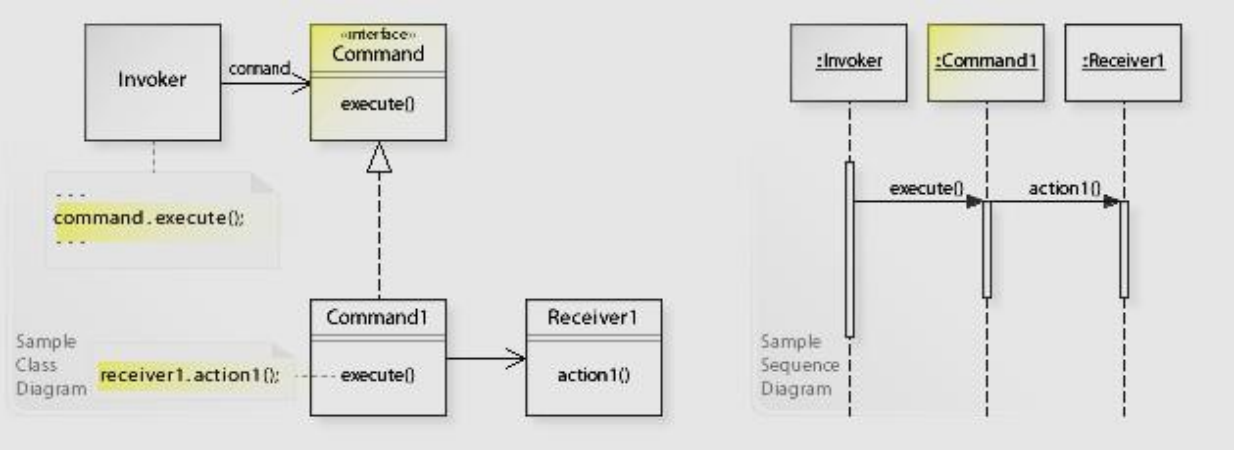
```
TextShape::TextShape (TextView* t)
 {
_text = t;
}
void TextShape::BoundingBox ( Point& bottomLeft, Point& topRight)
{
    Coord bottom, left, width, height;
      bottomLeft = Point(bottom, left);
      topRight = Point(bottom + height, left + width);
   }
Bool TextShape::IsEmpty ()
 {
      return _text->IsEmpty();
Manipulator*
TextShape::CreateManipulator ()
const { return new
TextManipulator(this);
}
```

3.10 Command

The command pattern is a behavioral design pattern in which an object is used to encapsulate all information needed to perform an action or trigger an event at a later time. This information includes the method name, the object that owns the method and values for the method parameters.

In the below UML class diagram, the Invoker class doesn't implement a request directly. Instead, Invoker refers to the Command interface to perform a request (command.execute()), which makes the Invoker independent of how the request is performed. The Command1 class implements the Command interface by performing an action on a receiver (receiver1.action1()).

The command design pattern describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.



Components of Command Pattern:

i. Command Classes

A command is an object whose role is to store all the information required for executing an action, including the method to call, the method arguments, and the object (known as the receiver) that implements the method.

ii. Receiver Class

A receiver is an object that performs a set of cohesive actions.

It's the component that performs the actual action when the command's execute() method is called.

iii. The Invoker Class

An invoker is an object that knows how to execute a given command but doesn't know how the command has been implemented. It only knows the command's interface. In some cases, the invoker also stores and queues commands, aside from executing them. This is useful for implementing some additional features, such as macro recording or undo and redo functionality.

iv. The Client Class

A client is an object that controls the command execution process by specifying what commands to execute and at what stages of the process to execute them.

Using the command design pattern can solve these problems:

- Coupling the invoker of a request to a particular request should be avoided. That is, hard-wired requests should be avoided.
- It should be possible to configure an object (that invokes a request) with a request.
- Using the command design pattern describes the following solution:
- Define separate (command) objects that encapsulate a request.
- A class delegates a request to a command object instead of implementing a particular request directly.

3.11 Strategy

Strategy pattern is a behavioral design pattern that allows the behavior of an object to be selected at runtime. The Strategy pattern is based on the idea of encapsulating a family of algorithms into separate classes that implement a common interface. The pattern consists of three main components: the Context, the Strategy, and the Concrete Strategy.

i. **The Context** is the class that contains the object whose behavior needs to be changed dynamically.

ii. **The Strategy** is the interface or abstract class that defines the common methods for all the algorithms that can be used by the Context object.

iii. **The Concrete** Strategy is the class that implements the Strategy interface and provides the actual implementation of the algorithm.

Also known as: policy

Motivation: Defining classes that encapsulate different line breaking algorithms. An algorithm that's encapsulated in this way is called a strategy.

Applicability: Use the Strategy pattern when

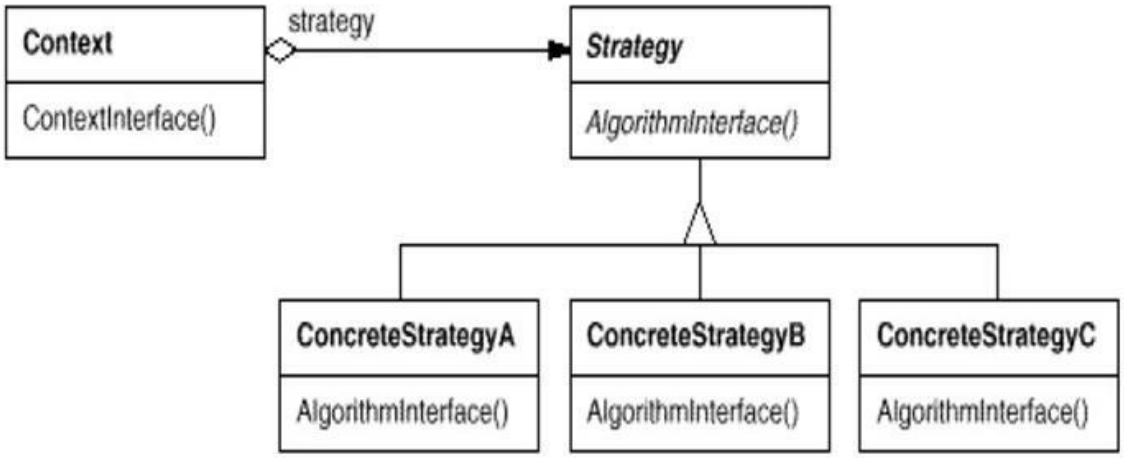
- many related classes differ only in their behavior.
- you need different variants of an algorithm.
- An algorithm uses data that clients shouldn't know about
- A class defines many behaviors, and these appear as multiple conditional statements in its operations.

Participants:

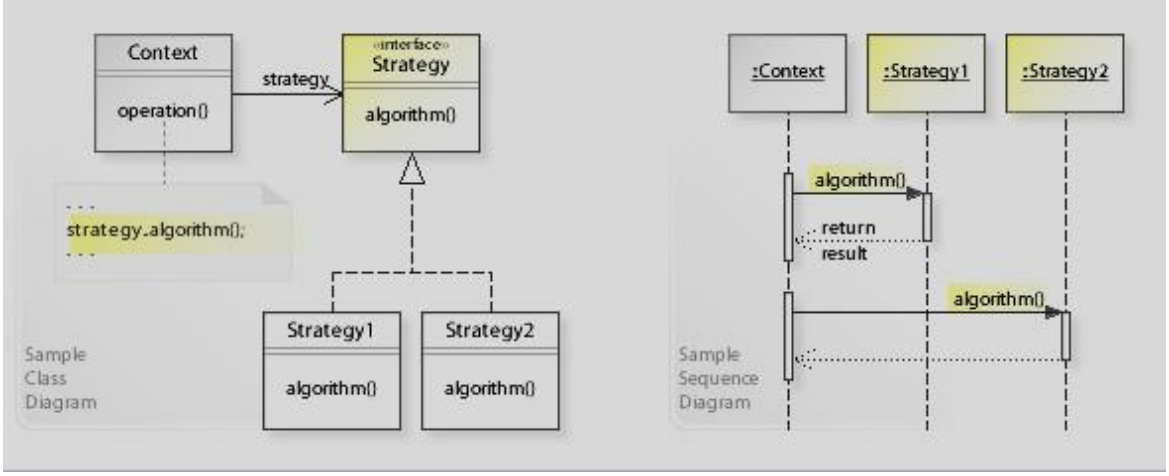
- Strategy -declares an interface common to all supported algorithms.
- ConcreteStrategy - implements the algorithm using the Strategy interface

- Context - maintains a reference to a Strategy object.

Structure:



In the below UML class diagram, the Context class doesn't implement an algorithm directly. Instead, Context refers to the Strategy interface for performing an algorithm (strategy.algorithm()), which makes Context independent of how an algorithm is implemented. The Strategy1 and Strategy2 classes implement the Strategy interface, that is, implement (encapsulate) an algorithm.The UML sequence diagram shows the run-time interactions: The Context object delegates an algorithm to different Strategy objects. First, Context calls algorithm() on a Strategy1 object, which performs the algorithm and returns the result to Context. Thereafter, Context changes its strategy and calls algorithm() on a Strategy2 object, which performs the algorithm and returns the result to Context



.**Collaborations:**

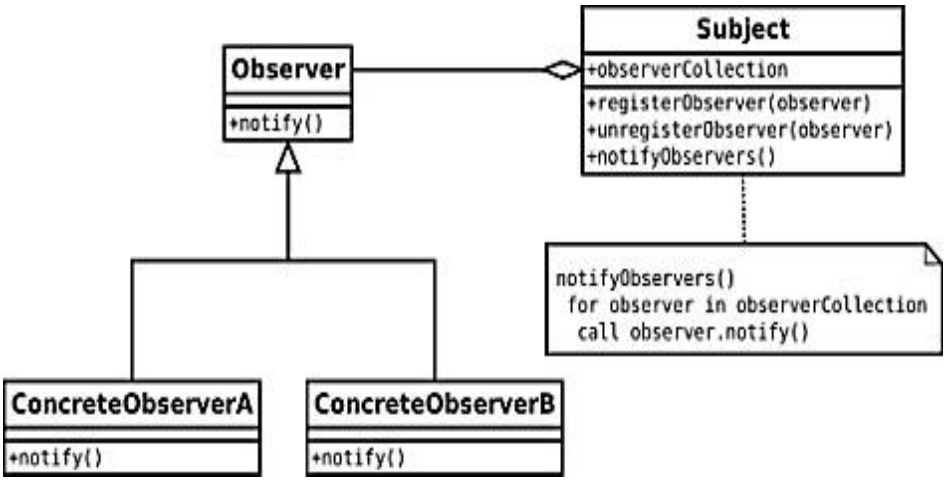
- Strategy and Context interact to implement the chosen algorithm.
- A context may pass all data required by the algorithm to the strategy when the algorithm is called.
- Alternatively, the context can pass itself as an argument to Strategy operations.
- A context forwards requests from its clients to its strategy.

Consequences:

- Families of related algorithms. Hierarchies of Strategy classes define a family of algorithms or behaviors for contexts to reuse. Inheritance can help factor out common functionality of the algorithms.
- An alternative to sub classing. Inheritance offers another way to support a variety of algorithms or behaviors.

3.12 Observer

The Observer Pattern is a behavioral design pattern that defines a one-to-many dependency between objects, meaning that when one object (the subject) changes its state, all its dependents (observers) are notified and updated automatically.

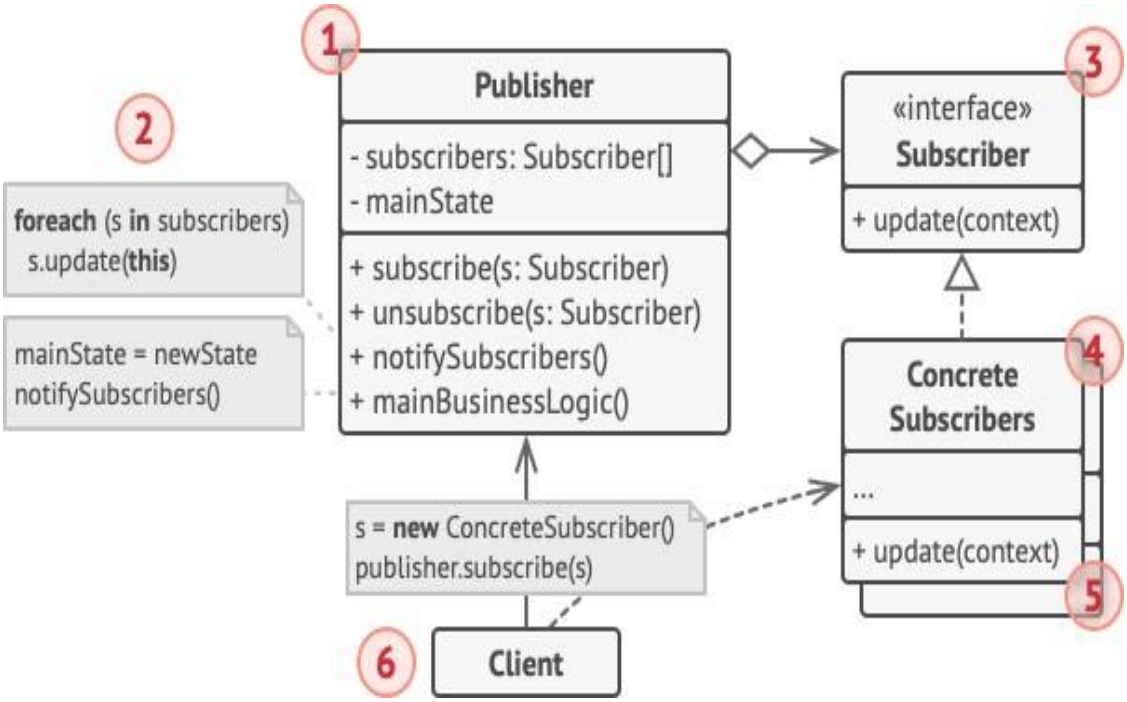


This pattern is used to build distributed event-handling systems and is a crucial part of many software architectures, including Model-View-Controller (MVC).

Intent: Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.

Structure:

1. The Publisher issues events of interest to other objects. These events occur when the publisher changes its state or executes some behaviors. Publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list.
2. When a new event happens, the publisher goes over the subscription list and calls the notification method declared in the subscriber interface on each subscriber object.
3. The Subscriber interface declares the notification interface. In most cases, it consists of a single update method.



4. Concrete Subscribers perform some actions in response to notifications issued by the publisher. All of these classes must implement the same interface so the publisher isn't coupled to concrete classes.

5. Usually, subscribers need some contextual information to handle the update correctly. For this reason, publishers often pass some context data as arguments of the notification method. The publisher can pass itself as an argument, letting subscriber fetch any required data directly.

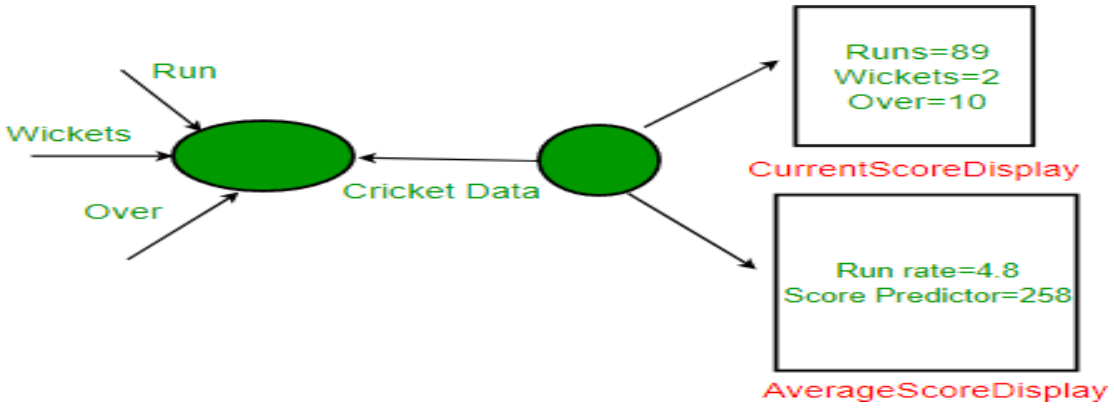
6. The Client creates publisher and subscriber objects separately and then registers subscribers for publisher updates.

Key Concepts of Observer Method Design Pattern:

The key concepts of the Observer Pattern are:

- Subject: The subject is the object that maintains a list of its dependents, also known as observers.It provides an interface for attaching, detaching, and notifying observers.
- Observer: The observer is the interface that defines the update method, which is called by the subject to notify the observer of a change in the subject's state.Observers register themselves with the subject to receive updates.
- ConcreteSubject: The concrete subject is a concrete implementation of the subject interface. It maintains the state of interest and notifies observers when a change occurs.
- ConcreteObserver: The concrete observer is a concrete implementation of the observer interface. It registers itself with a subject to receive updates and implements the update method to respond to changes in the subject's state.
- Observer List:The subject maintains a list (or collection) of observers, allowing multiple observers to be notified of changes.

Example: Suppose we are building a cricket app that notifies viewers about the information such as current score, run rate etc.



Suppose we have made two display elements CurrentScoreDisplay and AverageScoreDisplay. CricketData has all the data (runs, bowls etc.) and whenever data changes the display elements are notified with new data and they display the latest data accordingly.

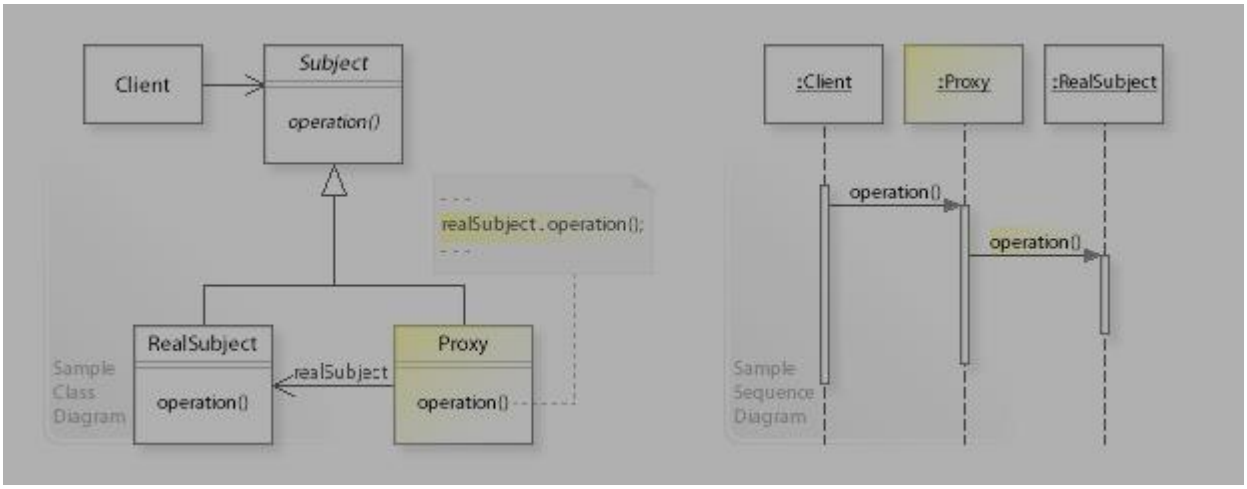
3.13 Proxy

A Proxy Method or Proxy Design Pattern is a structural design pattern that provides a surrogate or placeholder for another object to control access to it. This pattern involves creating a new class, known as the proxy, which acts as an intermediary between a client and the real object. The proxy object controls access to the real object.

In the real work a cheque or credit card is a proxy for what is in our bank account. It can be used in place of cash, which is what is needed, and provides a means of accessing that cash when required.

Also known as: surrogate

Architecture



In the above UML class diagram, the Proxy class implements the Subject interface so that it can act as substitute for Subject objects. It maintains a reference (realSubject) to the substituted object (RealSubject) so that it can forward requests to it (realSubject.operation()).The sequence diagram shows the run-time interactions: The Client object works through a Proxy object that controls the access to a RealSubject object. In this example, the Proxy forwards the request to the RealSubject, which performs the request.

3.13.1 Design participants

- **Subject –** is an interface which expose the functionality available to be used by the clients.

- **Real Subject** – is a class implementing Subject and it is concrete implementation which needs to be hidden behind a proxy.
- **Proxy** – hides the real object by extending it and clients communicate to real object via this proxy object. Usually frameworks create this proxy object when client request for real object.

3.13.2 Types of proxies

Proxies are generally divided into four types –

i. **Remote proxy** – represent a remotely lactated object. To talk with remote objects, the client need to do additional work on communication over network. A proxy object does this communication on behalf of original object and client focuses on real talk to do.

ii. **Virtual proxy** – delay the creation and initialization of expensive objects until needed, where the objects are created on demand. Hibernate created proxy entities are example of virtual proxies.

iii. **Protection proxy –** help to implement security over original object. They may check for access rights before method invocations and allow or deny access based on the conclusion.

iv. **Smart Proxy** – performs additional housekeeping work when an object is accessed by a client. An example can be to check if the real object is locked before it is accessed to ensure that no other object can change it.

3.14 Facade

Facade pattern hides the complexities of the system and provides an interface to the client using which the client can access the system. This type of design pattern comes under structural pattern as this pattern adds an interface to existing system to hide its complexities.

This pattern involves a single class which provides simplified methods required by client and delegates calls to methods of existing system classes.

The facade pattern is typically used when

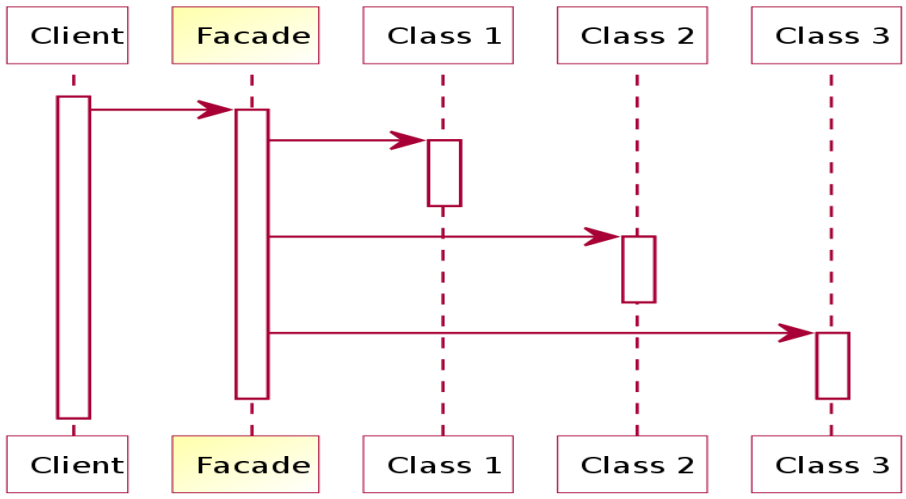
- a simple interface is required to access a complex system,
- a system is very complex or difficult to understand,
- an entry point is needed to each level of layered software, or
- the abstractions and implementations of a subsystem are tightly coupled.

Advantages

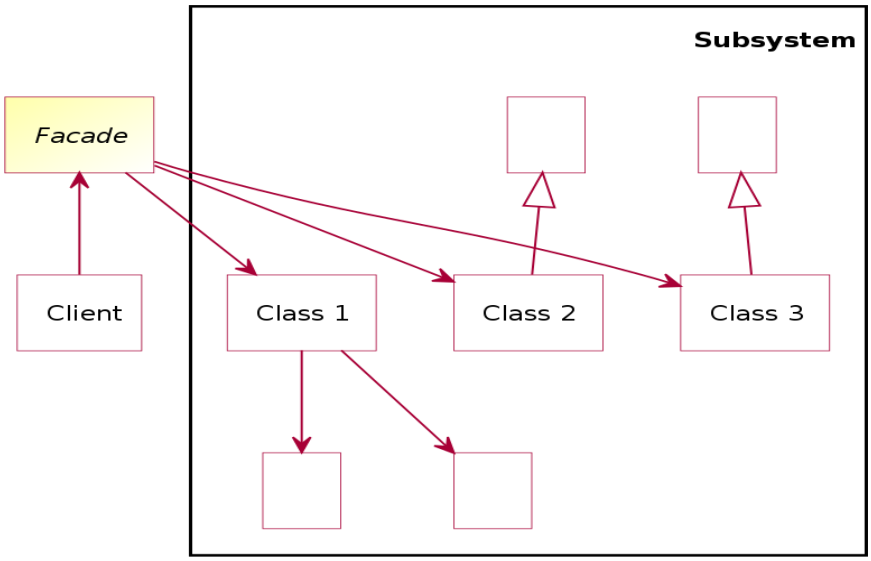
- It shields the clients from the complexities of the sub-system components.

- It promotes loose coupling between subsystems and its clients.

Structure:



Sample sequence diagram



Sample class diagram

This UML class diagram, the Client class doesn't access the subsystem classes directly. Instead, the Client works through a Facade class that implements a simple interface in terms of (by delegating to) the subsystem classes (Class1, Class2, and Class3).

The Client depends only on the simple Facade interface and is independent of the complex subsystem.The sequence diagram shows the run-time interactions: The Client object works through a Facade object that delegates the request to the Class1, Class2, and Class3 instances that perform the request.

3.15 Architectural style

The architectural style is a transformation and it is applied to the design of an entire system. The main aim of architectural style is to build a structure for all components of the system. Architecture of the system is redefined by using the architectural style. An architectural pattern such as architectural style introduces a transformation on the design of architecture. The software is constructed for computer based system and it shows one of the architectural styles from many of style. The design categories of architectural styles include:

(1) a set of components (e.g., a database, computational modules) that perform a function required by a system;

(2) a set of connectors that enable "communication, coordination and cooperation" among components;

(3) constraints that define how components can be integrated to form the system; and

(4) semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

Various Architecture styles are

i. Layered architectures
ii. Client Server style
iii. Tiered architecture
iv. Pipe and filter architecture

i. Layered architectures

In the layered architecture a number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set. At the outer layer, components service user interface operations. At the inner layer, components perform operating system interfacing. Intermediate layers provide utility services and application software functions. Once requirements engineering uncovers the characteristics and constraints of the system to be built, the architectural style and/or combination of patterns that best fits those characteristics and constraints

can be chosen. In many cases, more than one pattern might be appropriate and alternative architectural styles can be designed and evaluated

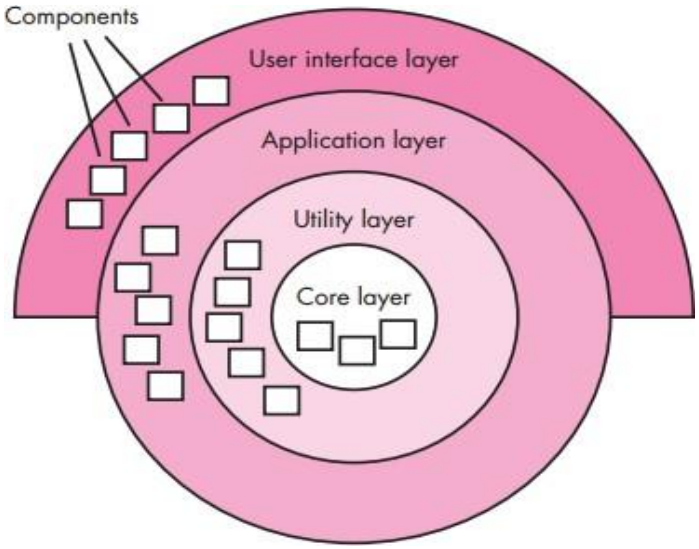


Figure 3.2: Layered architecture

ii. Client Server style

In the client/server architectural style (Figure a), a subsystem, the server, provides services to instances of other subsystems called the clients, which are responsible for interacting with the user. The request for a service is usually done via a remote procedure call mechanism or a common object broker (e.g., CORBA, Java RMI, or HTTP). Control flow in the clients and the servers is independent except for synchronization to manage requests or to receive results. Clients request services from one or more Servers. The Server has no knowledge of the Client. The client/server architectural style is a specialization of the repository architectural style. An information system with a central database is an example of a client/server architectural style. The clients are responsible for receiving inputs from the user, performing range checks, and initiating database transactions when all necessary data are collected.

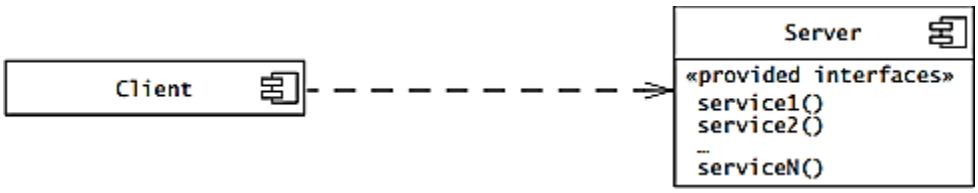


Figure 3.3 a: Client/server architectural style (UML component diagram).

The server is then responsible for performing the transaction and guaranteeing the integrity of the data. In this case, a client/server architectural style is a special case of the repository architectural style in which the central data structure is managed by a process. Client/server systems, however, are not restricted to a single server. On the World Wide Web, a single client can easily access data from thousands of different servers (figure a).

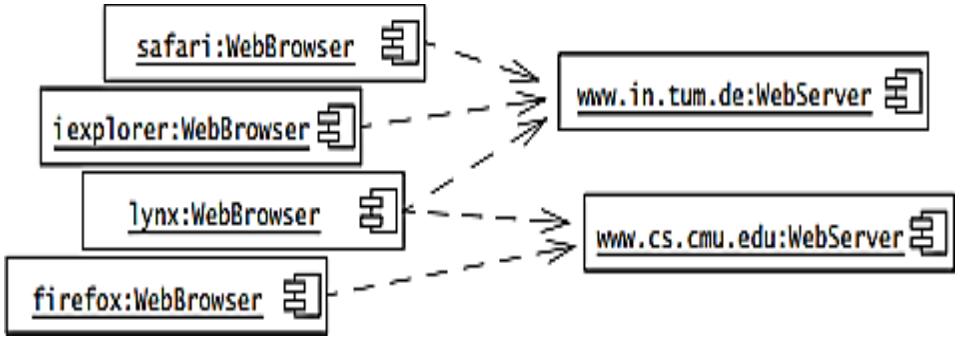


Figure3.3 b: The Web as an instance of the client/server architectural style
(UML deployment diagram).

Client/server architectural styles are well suited for distributed systems that manage large amounts of data.

iii. Tiered architecture

The three-tier architectural style organizes subsystems into three layers (Figure 3.4):

• The interface layer includes all boundary objects that deal with the user, including windows, forms, web pages, and so on.

• The application logic layer includes all control and entity objects, realizing the processing, rule checking, and notification required by the application.

• The storage layer realizes the storage, retrieval, and query of persistent objects.

Objects are organized into three layers realizing the user interface, the processing, and the storage. The three-tier architectural style was initially described in the 1970s for information systems. The storage layer, an analog to the Repository subsystem in the repository architectural style, can be shared by several different applications operating on the same data.

In turn, the separation between the interface layer and the application logic layer enables the development or modification of different user interfaces for the same application logic.

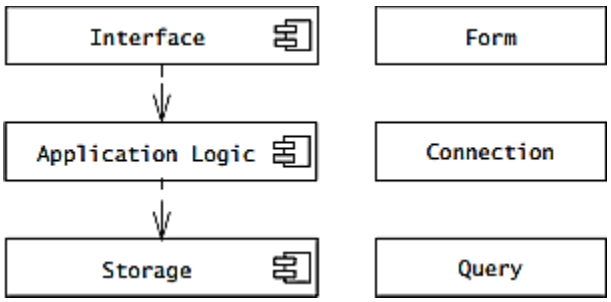


Figure 3.4: Three-tier architectural style (UML component diagram).

iv. Pipe and filter

In the pipe and filter architectural style , subsystems process data received from a set of inputs and send results to other subsystems via a set of outputs. The subsystems are called “filters,” and the associations between the subsystems are called “pipes.” Each filter knows only the content and the format of the data received on the input pipes, not the filters that produced them. Each filter is executed concurrently, and synchronization is accomplished via the pipes.

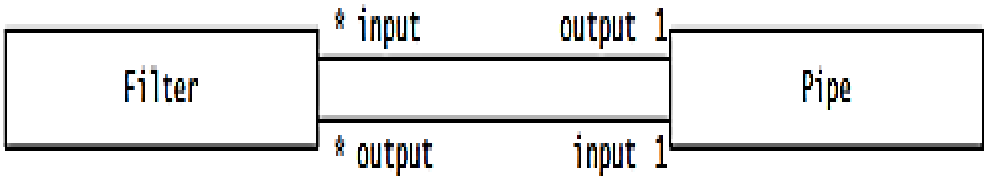


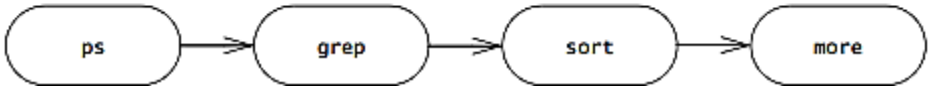
Figure 3.5: Pipe and filter architectural style (UML class diagram).

The pipe and filter architectural style is modifiable: filters can be substituted for others or reconfigured to achieve a different purpose.

A Filter can have many inputs and outputs. A Pipe connects one of the outputs of a Filter to one of the inputs of another Filter. The best known example of a pipe and filter architectural style is the Unix shell . Most filters are written such that they read their input and write their results on standard pipes. This enables a Unix user to combine them in many different ways. The following figure shows an example made of four filters. The output of ps (process status) is fed into grep (search for a pattern) to remove all the processes that are not owned by a specific user. The output of grep (i.e., the processes owned by the user) is then sorted by sort and sent to more, which is a filter that displays its input to a terminal, one screen at a time.

```
% ps auxwww | grep dutoit | sort | more

dutoit   19737  0.2  1.6 1908 1500 pts/6    O 15:24:36  0:00 -tcsh
dutoit   19858  0.2  0.7  816  580 pts/6    S 15:38:46  0:00 grep dutoit
dutoit   19859  0.2  0.6  812  540 pts/6    O 15:38:47  0:00 sort
```



Pipe and filter styles are suited for systems that apply transformations to streams of data without intervention by users. They are not suited for systems that require more complex interactions between components, such as an information management system or an interactive system.

3.16 User interface design

User interface design creates an effective communication medium between a human and a computer. User interface is the front-end application view to which user interacts in order to use the software. User can manipulate and control the software as well as hardware by means of user interface. Today, user interface is found at almost every place where digital technology exists, right from computers, mobile phones, cars, music players, airplanes, ships etc. User interface is part of software and is designed such a way that it is expected to provide the user insight of the software. UI provides fundamental platform for human-computer interaction. UI can be graphical, text-based, audio-video based, depending upon the underlying hardware and software combination. UI can be hardware or software or a combination of both. The software becomes more popular if its user interface is:

- Attractive
- Simple to use
- Responsive in short time
- Clear to understand
- Consistent on all interfacing screens

3.16.1 Three golden rules

i. Place the user in control.

ii. Reduce the user's memory load.

ii. Make the interface consistent.

These golden rules actually form the basis for a set of user interface design principles that guide this important aspect of software design.

i. Place the User in Control

- Define interaction modes in a way that does not force a user into unnecessary or undesired actions.- An interaction mode is the current state of the interface. For example, if spell check is selected in a word-processor menu, the software moves to a spell-checking mode. There is no reason to force the user to remain in spell-checking mode if the user desires to make a small text edit along the way. The user should be able to enter and exit the mode with little or no effort.
- Provide for flexible interaction- Because different users have different interaction preferences, choices should be provided. For example, software might allow a user to interact via keyboard commands, mouse movement, a digitizer pen, a multi touch screen, or voice recognition commands
- Allow user interaction to be interruptible and undoable- Even when involved in a sequence of actions, the user should be able to interrupt the sequence to do something else (without losing the work that had been done). The user should also be able to “undo” any action
- Streamline interaction as skill levels advance and allow the interaction to be customized- Users often find that they perform the same sequence of interactions repeatedly. It is worthwhile to design a “macro” mechanism that enables an advanced user to customize the interface to facilitate interaction.
- Hide technical internals from the casual user- The user interface should move the user into the virtual world of the application. The user should not be aware of the operating system, file management functions, or other arcane computing technology. In essence, the interface should never require that the user interact at a level that is “inside” the machine (e.g., a user should never be required to type operating system commands from within application software).
- Design for direct interaction with objects that appear on the screen- The user feels a sense of control when able to manipulate the objects that are necessary to perform a task in a manner similar to what would occur if the object were a physical thing. For example, an application interface that allows a user to “stretch” an object is an implementation of direct manipulation

ii. Reduce the User’s Memory Load

- Reduce demand on short-term memory- When users are involved in complex tasks, the demand on short-term memory can be significant. The interface should be designed to reduce the requirement to remember past actions, inputs, and results.
- Establish meaningful defaults- The initial set of defaults should make sense for the average user, but a user should be able to specify individual preferences.
- Define shortcuts that are intuitive- When mnemonics are used to accomplish a system function (e.g., alt-P to invoke the print function), the mnemonic should be tied to the action in a way that is easy to remember.
- The visual layout of the interface should be based on a real-world metaphor- For example, a bill payment system should use a checkbook and check register metaphor to guide the user through the bill paying process. This enables the user to rely on well-understood visual cues, rather than memorizing an arcane interaction sequence.
- Disclose information in a progressive fashion- The interface should be organized hierarchically. That is, information about a task, an object, or some behavior should be presented first at a high level of abstraction. More detail should be presented after the user indicates interest with a mouse pick.

iii. Make the Interface Consistent

The interface should present and acquire information in a consistent fashion. This implies that

(1) all visual information is organized according to design rules that are maintained throughout all screen displays,

(2) input mechanisms are constrained to a limited set that is used consistently throughout the application, and

(3) mechanisms for navigating from task to task are consistently defined and implemented.

- Allow the user to put the current task into a meaningful context.
- Maintain consistency across a family of applications.

3.17 User interface analysis and design

The overall process for analyzing and designing a user interface begins with the creation of different models of system function. Tools are used to prototype and ultimately implement the design model, and the result is evaluated by end users for quality.

3.17.1 Design Models

Four different models come into play when a user interface is to be analyzed and designed. A human engineer (or the software engineer) establishes a user model, the software engineer creates a design model, the end user develops a mental image that is often called the user's mental model or the system perception, and the implementers of the system create an implementation model. Unfortunately, each of these models may differ significantly.

In addition, users can be categorized as:

i. Novices. No syntactic knowledge1 of the system and little semantic knowledge2 of the application or computer usage in general.

ii. Knowledgeable, intermittent users. Reasonable semantic knowledge of the application but relatively low recall of syntactic information necessary to use the interface.

iii. Knowledgeable, frequent users. Good semantic and syntactic knowledge that often leads to the "power-user syndrome"; that is, individuals who look for shortcuts and abbreviated modes of interaction.

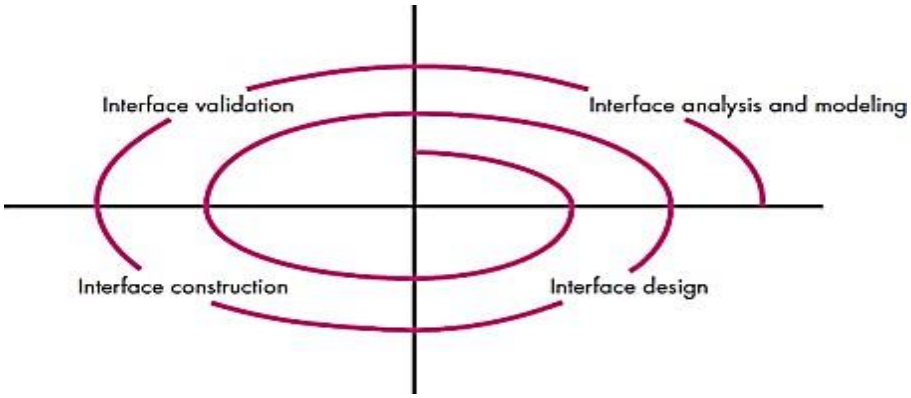
The Process: The analysis and design process for user interfaces is iterative and can be represented using a spiral model.The four distinct framework activities:

(1) interface analysis and modeling,

(2) interface design,

(3) interface construction, and

(4) interface validation. The spiral model implies that each of these tasks will occur more than once, with each pass around the spiral representing additional elaboration of requirements and the resultant design.



The goal of interface design is to define a set of interface objects and actions (and their screen representations) that enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system. Interface construction normally begins with the creation of a prototype that enables usage scenarios to be evaluated. As the iterative design process continues, a user interface tool kit may be used to complete the construction of the interface. Interface validation focuses on

(1) the ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements;

(2) the degree to which the interface is easy to use and easy to learn, and

(3) the users' acceptance of the interface as a useful tool in their work. Subsequent passes through the process elaborate task detail, design information, and the operational features of the interface.

3.17.2 Interface analysis

A key tenet of all software engineering process models is this: understand the problem before you attempt to design a solution. In the case of user interface design, understanding the problem means understanding

(1) the people (end users) who will interact with the system through the interface,

(2) the tasks that end users must The information gathered as part of the analysis action is used to create an analysis perform to do their work,

(3) the content that is presented as part of the interface, and (4) the environment in which these tasks will be conducted.

User Analysis

The phrase "user interface" is probably all the justification needed to spend some time understanding the user before worrying about technical matters. In addition, the user's mental image may be vastly different from the software engineer's design model. The only way that you can get the mental image and the design model to converge is to work to understand the users themselves as well as how these people will use the system. Information from a broad array of sources can be used to accomplish this:

User Interviews: The most direct approach, members of the software team meet with end users to better understand their needs, motivations, work culture, and a myriad of other issues. This can be accomplished in one-on-one meetings or through focus groups.

Sales input: Sales people meet with users on a regular basis and can gather information that will help the software team to categorize users and better understand their requirements.

Marketing input: Market analysis can be invaluable in the definition of market segments and an understanding of how each segment might use the software in subtly different ways.

Support input: Support staff talks with users on a daily basis. They are the most likely source of information on what works and what doesn't, what users like and what they dislike, what features generate questions and what features are easy to use.

Task Analysis and Modeling

The goal of task analysis is to answer the following questions:

- What work will the user perform in specific circumstances?
- What tasks and subtasks will be performed as the user does the work?
- What specific problem domain objects will the user manipulate as work is performed?
- What is the sequence of work tasks—the workflow?
- What is the hierarchy of tasks?

Use cases

When used as part of task analysis, the use case is developed to show how an end user performs some specific work-related task. In most instances, the use case is written in an informal style (a simple paragraph) in the first-person. For example, assume that a small software company wants to build a computer-aided design system explicitly for interior designers. To get a better understanding of how they do their work, actual interior designers are asked to describe a specific design function. When asked: "How do you decide where to put furniture in a room?" an interior designer writes the following informal use case:

This use case provides a basic description of one important work task for the computer-aided design system. From it, you can extract tasks, objects, and the overall flow of the interaction. In addition, other features of the system that would please the interior designer might also be conceived.

Task elaboration

The stepwise elaboration (also called functional decomposition or stepwise refinement) as a mechanism for refining the processing tasks that is required for software to accomplish some

desired function. Task analysis for interface design uses an elaborative approach to assist in understanding the human activities the user interface must accommodate.

Object elaboration

Rather than focusing on the tasks that a user must perform, you can examine the use case and other information obtained from the user and extract the physical objects that are used by the interior designer. These objects can be categorized into classes. Attributes of each class are defined, and an evaluation of the actions applied to each object provide a list of operations. For example, the furniture template might translate into a class called Furniture with attributes that might include size, shape, location, and others. The interior designer would select the object from the Furniture class, move it to a position on the floor plan (another object in this context), draw the furniture outline, and so forth. The tasks select, move, and draw are operations. However, as the design is elaborated, the details of each operation are defined.

Workflow analysis

When a number of different users, each playing different roles, makes use of a user interface, it is sometimes necessary to go beyond task analysis and object elaboration and apply workflow analysis. This technique allows you to understand how a work process is completed when several people (and roles) are involved. Consider a company that intends to fully automate the process of prescribing and delivering prescription drugs. The entire process will revolve around a Web-based application that is accessible by physicians (or their assistants), pharmacists, and patients. Workflow can be represented effectively with a UML swimlane diagram (a variation on the activity diagram).

Hierarchical representation

A process of elaboration occurs as you begin to analyze the interface. Once workflow has been established, a task hierarchy can be defined for each user type. The hierarchy is derived by a stepwise elaboration of of the actions applied to each object provide a list of operations. For example, the furniture template might translate into a class called Furniture with attributes that might include size, shape, location, and others. The interior designer would select the object from the Furniture class, move it to a position on the floor plan (another object in this context), draw the furniture outline, and so forth. The tasks select, move, and draw are operations. However, as the design is elaborated, the details of each operation are defined.

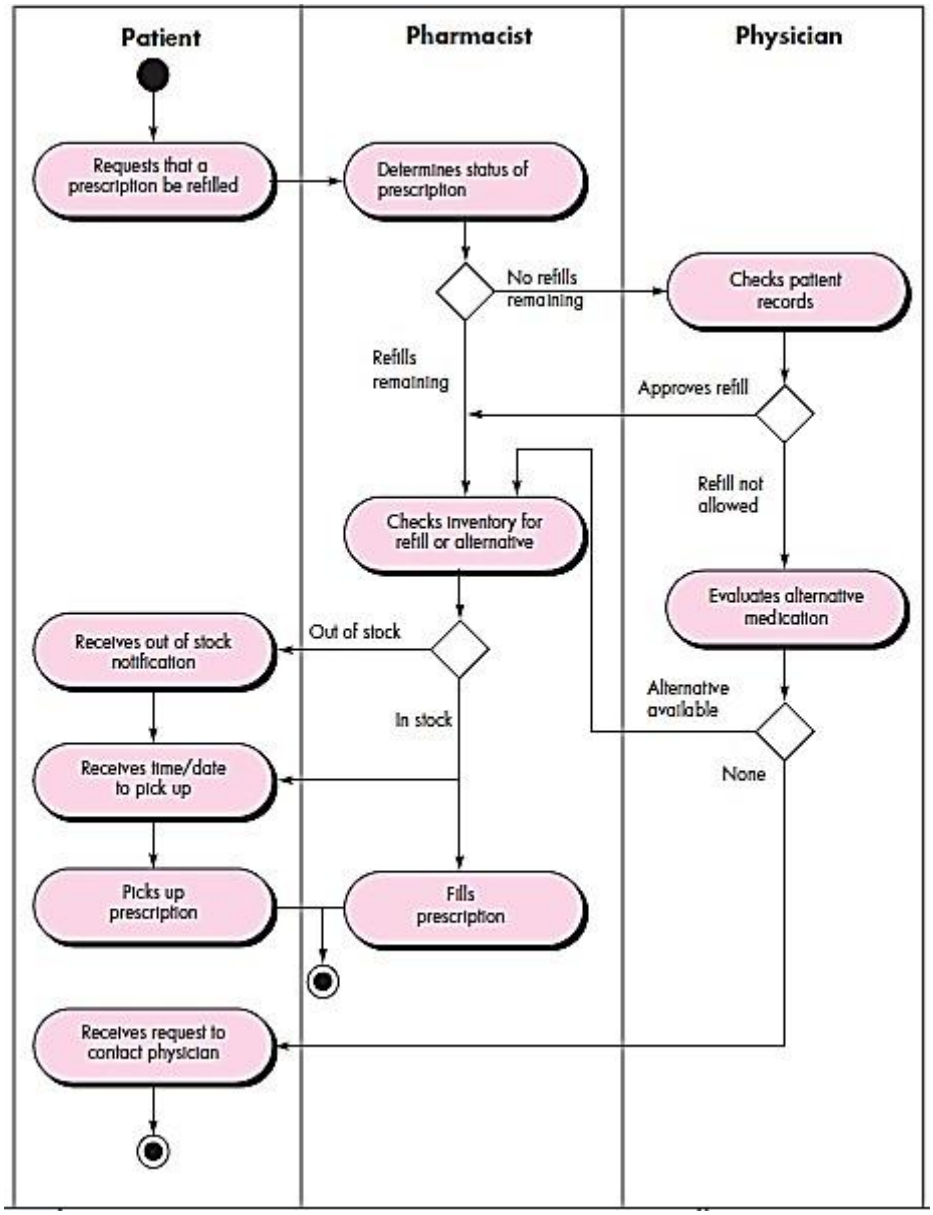


Figure 3.6: Swim lane diagram for prescription refill function

Analysis of the Work Environment

People do not perform their work in isolation. They are influenced by the activity around them, the physical characteristics of the workplace, the type of equipment they are using, and the work relationships they have with other people. If the products you design do not fit into the environment, they may be difficult or frustrating to use. In some applications the user interface for a computer-based system is placed in a “user-friendly location” (e.g., proper lighting, good display height, easy keyboard access), but in others (e.g., a factory floor or an airplane cockpit), lighting

may be suboptimal, noise may be a factor, a keyboard or mouse may not be an option, display placement may be less than ideal. The interface designer may be constrained by factors that mitigate against ease of use. In addition to physical environmental factors, the workplace culture also comes into play.

Interface design steps

Once interface analysis has been completed, all tasks (or objects and actions) required by the end user have been identified in detail and the interface design activity commences. Interface design, like all software engineering design, is an iterative process. Each user interface design step occurs a number of times, elaborating and refining information developed in the preceding step. Although many different user interface design models have been proposed, all suggest some combination of the following steps:

1. Using information developed during interface analysis, define Interface objects and actions (operations).
2. Define events (user actions) that will cause the state of the user interface to change. Model this behavior.
3. Depict each interface state as it will actually look to the end user.
4. Indicate how the user interprets the state of the system from information provided through the interface.

User Interface Design Patterns

Graphical user interfaces have become so common that a wide variety of user interface design patterns has emerged.

There are many possible solutions to this simple problem, and a number of different patterns that might be proposed.

Design Issues:

As the design of a user interface evolves, four common design issues almost always surface:

i. system response time,
ii. user help facilities,
iii. error information handling,
iv. menu and command labeling.

i. **Response time:** System response time is the primary complaint for many interactive applications. In general, system response time is measured from the point at which the user performs some control action (e.g., hits the return key or clicks a mouse) until the software responds with desired output or action.

System response time has two important characteristics: length and variability. If system response is too long, user frustration and stress are inevitable. Variability refers to the deviation from average response time, and in many ways, it is the most important response time characteristic.

ii. **Help facilities:** Almost every user of an interactive, computer-based system requires help now and then. In some cases, a simple question addressed to a knowledgeable colleague can do the trick. In others, detailed research in a multivolume set of "user manuals" may be the only option. In most cases, however, modern software provides online help facilities that enable a user to get a question answered or resolve a problem without leaving the interface..

iii. **Error handling:** Error messages and warnings are "bad news" delivered to users of interactive systems when something has gone awry. At their worst, error messages and warnings impart useless or misleading information and serve only to increase user frustration. There are few computer users who have not encountered an error of the form.In general, every error message or warning produced by an interactive system should have the following characteristics:

- The message should describe the problem in jargon that the user can understand.
- The message should provide constructive advice for recovering from the error.
- The message should indicate any negative consequences of the error (e.g., potentially corrupted data files) so that the user can check to ensure that they have not occurred (or correct them if they have).

iv. **Menu and command labeling**

The typed command was once the most common mode of interaction between user and system software and was commonly used for applications of every type. Today, the use of window-oriented, point-and pick interfaces has reduced reliance on typed commands, but some power-users continue to prefer a command-oriented mode of interaction. A number of design issues arise when typed commands or menu labels are provided as a mode of interaction:

- Will every menu option have a corresponding command?

- Can commands be customized or abbreviated by the user?
- Are menu labels self-explanatory within the context of the interface?
- Are submenus consistent with the function implied by a master menu item?

Part A (Two Marks)

1. What are the elements of design model?

i. Data design

ii. Architectural design

iii. Interface design

iv. Component-level design

2. Define design process.

Design process is a sequence of steps carried through which the requirements are translated into a system or software model.

3. List the principles of a software design.

i. The design process should not suffer from “tunnel vision”

ii. The design should be traceable to the analysis model.

iii. The design should exhibit uniformity and integration.

iv. Design is not coding.

v. The design should not reinvent the wheel.

4. What is the benefit of modular design?

Changes made during testing and maintenance becomes manageable and they do not affect other modules.

5. What is a cohesive module?

A cohesive module performs only “one task” in software procedure with little interaction with other modules. In other words cohesive module performs only one thing.

6. What are the different types of Cohesion?

i. Coincidentally cohesive - The modules in which the set I\of tasks are related with each other loosely.

ii. Logically cohesive – A module that performs the tasks that are logically related with each other.

iii. Temporal cohesion – The module in which the tasks need to be executed in some specific time span.

iv. Procedural cohesion – When processing elements of a module are related with one another and must be executed in some specific order.

v. Communicational cohesion – When the processing elements of a module share the data then such module is called communicational cohesive.

7. What is coupling?

Coupling is the measure of interconnection among modules in a program structure. It depends on the interface complexity between modules.

8. What are the various types of coupling?

i. Data coupling – The data coupling is possible by parameter passing or data interaction.

ii. Control coupling – The modules share related control data in control coupling.

iii. Common coupling – The common data or a global data is shared among modules.

iv. Content coupling – Content coupling occurs when one module makes use of data or control information maintained in another module.

9. What are the common activities in design process?

i. Modular decomposition – The identified subsystems are decomposed into modules.

ii. System structuring – The system is subdivided into principle subsystems components and communications between these subsystems are identified.

iii. Control modeling – A model of control relationships between different parts of the system is established.

10. What are the benefits of horizontal partitioning?

i. Software that is easy to test.

ii. Software that is easier to maintain.

iii. Propagation of fewer side effects.

iv. Software that is easier to extend.

11. What is vertical partitioning?

Vertical partitioning often called factoring suggests that the control and work should be distributed top-down in program structure.

12. What are the advantages of vertical partitioning?

i. These are easy to maintain changes.

ii. They reduce the change impact and error propagation.

13. What are the various elements of data design?

i. Data object – The data objects are identified and relationship among various data objects can be represented using ERD or data dictionaries.

ii. Databases – Using software design model, the data models are translated into data structures and data bases at the application level.

iii. Data warehouses – At the business level useful information is identified from various databases and the data warehouses are created.

14. List the guidelines for data design.

i. Apply systematic analysis on data.

ii. Identify data structures and related operations.

iii. Establish data dictionary.

iv. Use information hiding in the design of data structure.

v. Apply a library of useful data structures and operations.

15. Name the commonly used architectural styles.

i. Data centered architecture.

ii. Data flow architecture.

iii. Call and return architecture.

iv. Object-oriented architecture.

v. Layered architecture.

16. What is Transform mapping?

The transform mapping is a set of design steps applied on the DFD in order to map the transformed flow characteristics into specific architectural style.

17. What is a Real time system?

Real time system is a software system in which the correct functionalities of the system are dependent upon results produced by the system and the time at which these results are produced.

18. What are the objectives of Analysis modeling?

i. To describe what the customer requires.

ii. To establish a basis for the creation of software design.

iii. To devise a set of valid requirements after which the software can be built.

19. What is an Architectural design?

The architectural design defines the relationship between major structural elements of the software, the “design patterns” that can be used to achieve the requirements that have been defined for the system.

20. What is data design?

The data design transforms the information domain model created during analysis into the data structures that will be required to implement the software.

21. What is interface design?

The interface design describes how the software communicates within itself, with systems that interoperate with it, and with humans who use it.

22. What is component level design?

The component level design transforms structural elements of the software architecture into a procedural description of software components.

23. What is software design?

Software design is an iterative process through which the requirements are translated into a “blueprint” for constructing the software.

24. What is user interface design?

User interface design creates an effective communication medium between a human and a computer.

25. What is system design?

System design process involves deciding which system capabilities are to be implemented in software and which in hardware.

26. Write notes on FURPS model of design quality?

Functionality: Evaluating the feature set and capabilities of the program.

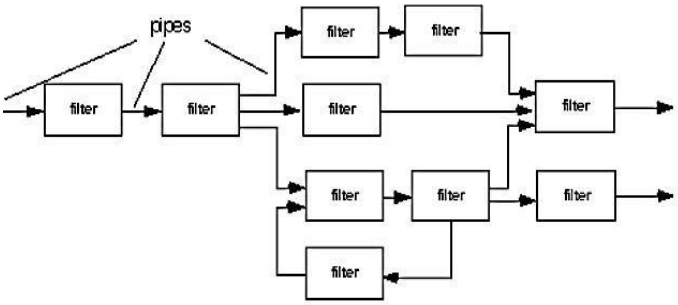
Usability: Ease of use.

Reliability: Ability to recover from failure.

Performance: Measured by processing speed, response time, throughput, efficiency.

Supportability: Ability to extend the program.

27. Draw diagrams to demonstrate the architectural styles.



(a) pipes and filters

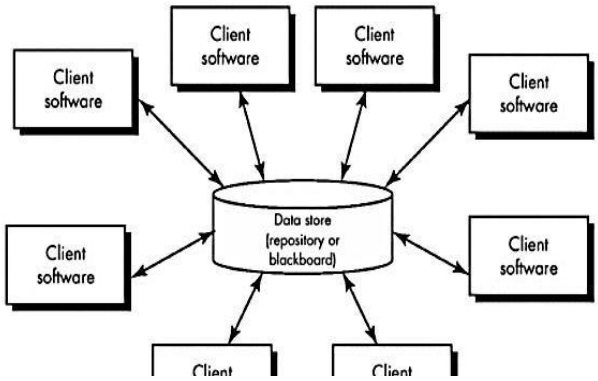


Fig. Data Centered Architecture

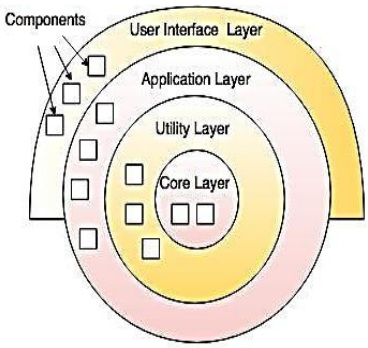


Fig.- Layered Architecture

28. Define patterns.

A pattern is a named problem/solution pair that can be applied in new context, with advice on how to apply it in novel situations and discussion of its trade-offs.

29. List out GRASP Patterns?

- Information Expert .
- Creator .
- High Cohesion .
- Low Coupling .
- Controller

30. What is adapter pattern?

The adapter pattern is a design pattern that translates one interface for a class into a compatible interface. An adapter allows classes to work together that normally could not because of incompatible interfaces, by providing its interface to clients while using the original interface. The adapter is also responsible for transforming data into appropriate forms.

31. What is Facade Pattern?

A facade is an object that provides a simplified interface to a larger body of code, such as a class library. A facade can:

- make a software library easier to use, understand and test, since the facade has convenient methods for common tasks;
- make code that uses the library more readable, for the same reason;
- reduce dependencies of outside code on the inner workings of a library, since most code uses the facade, thus allowing more flexibility in developing the system;
- Wrap a poorly-designed collection of APIs with a single well-designed API

Part B (Possible Questions)

1. Explain in detail the design concepts.
2. Explain the design model.
3. Explain the User Interface Design.
4. Explain in detail about the designing components.
5. Explain in detail about the user interface design activities.
6. Explain data, architectural and procedural design principles for the software.
7. Describe the design procedures for the data acquisition system.
8. Explain in detail about the real time systems.
9. Explain in detail about Software Configuration Management (SCM).

Part C (Possible Questions)

1. Explain how the interrupts handled in real time systems?
2. Explain the interface design activities. What steps do we perform to accomplish Interface design?
3. Explain the design considerations for real time systems.
4. Explain about the component level design.
5. Describe about the designing class based components &traditional components.
6. Write elaborates about architectural design.
7. Explain about Data Acquisition Systems.
8. Explain GRASP: designing objects with responsibilities.
9. Explain in detail about Adaptor, Proxy, Model View Controller.